

Assignment #1 — Builder Pattern

Course: ShP-2216 – Software Design Patterns (OP 6B06102)

Institution: Astana IT University — School of Computer Engineering

Type: Individual assignment

Weight: 15% of the 1st Attestation grade

Language: Java (JDK 17, IntelliJ IDEA recommended)

Submission deadline: Sunday, 11:59 PM — end of Week 1

Defense date: Last practice class of Week 2

1. Learning Objectives

- Explain the purpose, structure, and applicability of the Builder pattern (CLO 1).
- Implement a correct, well-structured Builder pattern solution in Java that follows clean coding standards (CLO 2).
- Defend and justify your design decisions in front of the instructor (CLO 3).

2. Task Description

Design and implement the Builder creational design pattern in Java.

Free topic

- Propose your own product/domain, provided it genuinely benefits from step-by-step construction and multiple representations (see Section 3). Briefly confirm free topics with the instructor before you start implementing.

3. Technical Requirements

Component	Requirement
Product	A class representing the complex object being built (e.g., Car, Report, Pizza).
Builder	An interface or abstract class declaring the construction steps.
ConcreteBuilder(s)	At least two concrete builders producing meaningfully different representations of the product.
Director (optional, encouraged)	Orchestrates the build sequence for known/reusable configurations.
Client	A main method or small demo/test class that exercises the builders and shows the resulting products.

Method chaining (fluent API — each setter returns the builder itself) is expected, consistent with what was covered in lecture.

4. Clean Code Requirements

In your report, explicitly identify and justify at least five Clean Code principles applied in your implementation, for example:

- Meaningful, intention-revealing names for classes/methods/variables
- Small methods, each doing one thing
- No duplicated construction logic between builders

- Minimal, purposeful comments (code that explains itself vs. comments that explain why)
- Consistent formatting and small, focused classes
- Validated construction (e.g., `build()` throws a clear exception on invalid state)
- No magic numbers/strings

For each principle, include a short before/after or annotated code excerpt — not just a bullet-point claim.

5. Deliverables

A. GitHub Repository

- Complete, compiling source code
- Clear folder structure and a short README.md (what the product is, how to build each representation, how to run it)
- Incremental commit history (not a single "final" commit)

B. Report (Moodle LMS, before the defense)

- 1. Introduction — chosen product/topic and why Builder fits
- 2. UML class diagram of your implementation (Product, Builder, ConcreteBuilder(s), Director if used, Client) — PlantUML or UMLet recommended
- 3. Clean Code section — the 5+ principles with code excerpts (Section 4)
- 4. Conclusion — pros/cons you personally encountered
- 5. Link to your GitHub repository

C. In-Class Defense

- Verbal walkthrough of your design decisions
- Live (or screen-recorded) demonstration of the code running
- Answering instructor questions on the pattern and your implementation choices

6. Grading (100% of this assignment → counted as 15% of the 1st Attestation)

Component	Weight	What is assessed
Report	10%	Clarity, structure, completeness; correct UML diagram; Clean Code section substantiated with real examples
Working & Clean Code	40%	Functional correctness of the Builder implementation; readability, maintainability, adherence to Clean Code best practices
Defense	50%	Demonstrated understanding, clarity of explanation, ability to answer questions on the pattern and design choices

Graded on the standard course rubric (Excellent 90–100 / Good 70–89 / Satisfactory 50–69 / Fail 0–49).

7. Submission & Deadline

- **Submission deadline:** Sunday, 11:59 PM, end of Week 1. Upload your GitHub repository link and report to Moodle by this time.
- **Defense deadline:** last practice class of Week 2.
- Late submissions: per course policy, any submission made after the Sunday 11:59 PM deadline receives 0, unless a prior extension was approved for a documented exceptional circumstance.

Note: Week numbers correspond to the official Moodle-published course calendar — confirm the exact calendar dates there.

8. Academic Integrity

This is an individual assignment. Discussing concepts with classmates is encouraged, but all submitted code and report content must be your own. Plagiarism or code copying will be handled per university policy and may result in a zero grade and disciplinary referral.

9. Resources

- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship, Ch. 3
- Freeman, E., & Robson, E. Head First Design Patterns — Builder chapter
- Lecture 1 slides (Moodle LMS) — Builder pattern components and steps
- refactoring.guru/design-patterns/builder
- PlantUML / UMLet — for the UML diagram